Listing 7.13 Adding validation to minimal APIs using MinimalApis.Extensions

```
using System.ComponentModel.DataAnnotations;

WebApplicationBuilder builder = WebApplication.CreateBuilder(args);
WebApplication app = builder.Build();

app.MapPost("/user/{id}",
    ([AsParameters] GetUserModel model) => $"Received {model.Id}")  ❶
     .WithParameterValidation();                                    ❷

app.Run();

struct GetUserModel
{
    [Range(1, 10)]                                                  ❸
    Public int Id { get; set; }                                     ❸
}
```

❶ Uses [AsParameters] to create a type than can be validated

❷ Adds the validation filter to the endpoint

❸ Adds validation attributes to your simple types

That concludes this look at model binding in minimal APIs. You saw how the ASP.NET Core framework uses model binding to simplify the process of extracting values from a request and turning them into normal .NET objects you can work with quickly. The many ways to bind may be making your head spin, but normally, you can stick to the basics and fall back to the more complex types as and when you need them.

Although the discussion is short, the most important aspect of this chapter is its focus on validation—a common concern for all web applications. Whether you choose to use `DataAnnotations` or a different validation approach, you must make sure to validate any data you receive in all your endpoints.